Recycling Sorting Neural Networks

Jacob Diaz
FIU Researcher
Florida International University
line 4: Broward, USA
jediaz@engineer.com

***Abstract*—This project was done on limited hardware.**

I. Introduction (*Heading 1*)

Recycling is a requirement for the future sustainability of this world. The issue is there is little to no profitability in recycling. One of the largest expenses is the sorting of the recycling trash. Still much of the sorting is manually done which is where most of the cost comes from. To combat this, a model should be developed to automatically identify the recyclable item. This is where deep learning neural networks come into play. To design a custom neural network that identifies recyclable types is the goal.

II. Problem

The problem behind sorting recycling is there are many types of materials that can be recycled, just not together. However each material can have a huge variety of appearances (color, shape, texture etc.) that can overlap. The overall shape and color should be taken into account when sorting. Which is what makes it a task that needs human supervision.

III. Motivation

To cut on these costs is to make recycling more affordable/profitable, which in turn results in more recycling which is better for the environment as a whole. Using an AI model that can identify trash types and sort them can cut costs on recycling.

IV. Literature Review

A total of 7 different types of layers were used in the creation of these two models.

1. Convolution 2D layer [4]
    a. Applies a dot product along with an added weight over a section of the input and repeats until the whole input is processed.
2. BatchNormalization layer [2]
    a. Normalizes the inputs which creates a stable learning environment
3. MaxPooling layer [1]
    a. Goes over a section of the input and selects from them the largest value. It repeats this process until the whole input has been gone over
    b. This preserves spatial features while shrinking the size of the input
4. Dense layer [3]
    a. Similar idea to the convolution layer except instead of applying itself over sections at a time the whole input is processed at once.
    b. It can be used the shrink or increase the amount of hidden units the model uses
5. Dropout
    a. Temporarily deactivates a percentage of neurons which allows for a more generalized idea
    b. Helps prevent overfitting by creating a base understanding of something that doesn't focus on a single feature
6. Flatten
    a. Takes a multi dimensional input and converts it into a one dimensional array
    b. Required for Dense layers if the input isn't one dimensional
7. MobileNetV2 [5]
    a. Less of a single layer and more of an already trained model with all its layers
    b. We can utilize the trained layers and retrain them on a little bit of data that way we can save a lot of training
    c. For this model we froze the weights which meant that we didn't train the base model only the layers after

V. Dataset Information

The dataset used is Suman Kunwar's garbage dataset[6]. It includes 3 different datasets, "original" which is not standardized and has images of varying size, "standardized_256" which resizes all images to 256x256 pixels, and lasts "standardized_512" which resizes all images to a 512x512 pixel size. Each of the datasets contain 10 categories, shoes, cardboard, plastic, trash, glass, biological, clothes, battery, paper and metal. The categories are separated by folders with names according to the category type and in those folders contain all images in their category. Most categories contain about 1,500 images each. However a few do contain, by comparison low count, those being battery, biological, metal and trash all which contain from 400-1000 images. We also removed trash as the folder did contain plastics inside.

VI. Feature Processing and Feature Engineering

A. List and Array Structures

There are 6 lists in total, 4 of which later get transformed into an array. The lists are xData, yData, xTrain, yTrain, xTest, yTest.

- xData contains every image in a blue green red (bgr) format per pixel. This results in a shape like (samples, width, height, bgr).
- yData contains per xData image a 9 element long list which has zeros in every index except the corresponding image classification for xData's sample, the other index contains a one. This is a process known as one hot encoding. We identify each image classification by which folder it is held in.

- xData is later split into xTrain and xTest. Where xTrain has 80% of the xData images randomly shuffled in. While xTest contains 20% of the other images. These two lists are converted later into an array with numpy.
    - yTrain and yTest also contain a shuffled fraction of yData. However they are shifted in the same order xTrain and xTest are, to keep the correct classification to each image. These two lists are also converted into an array using numpy.

B. Image Processing

The original dataset was used because we will resize the images ourselves. The opencv python library was used to convert each image into a 3D list with a shape of (width, height, bgr).

Then the image's width and height are resized using opencv into a 32x32 pixel shape to save computer processing power later on in training. The 3D list is captured and appended into another list for a shape of (samples, width, height, bgr).

C. Image Augmentation

Since a huge disparity in image count per class was found for some classifications, images from the standardized 256 dataset were mixed into the under-represented classes to even out the classes' image representations.

To create more images for the model to train on we created rotated duplicates of each image in the xTrain array and added them with our model and then reshuffled the xTrain array to prevent a bias by order. This essentially quadrupled our data we could train on.

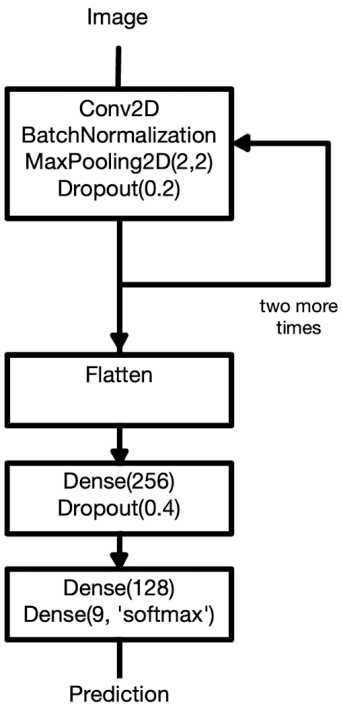
VII. Model Building

Two models were tested for this. A custom built convolution neural network, and we trained another model MobileNetV2 on our data set using transfer learning.

A. Custom CNN Model

The custom build sequential model is built in this order

1. Conv2D
   BatchNormalize
   MaxPooling(2,2)
   Dropout(0.2)
2. Conv2D
   BatchNormalize
   MaxPooling(2,2)
   Dropout(0.2)
3. Conv2D
   BatchNormalize
   MaxPooling(2,2)
   Dropout(0.2)
4. Flatten
5. Dense(256)
   Dropout(0.4)
6. Dense(128)
   Dense(9, softmax)

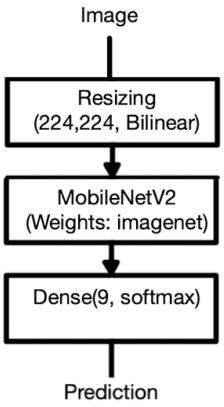


B. MobileNetV2 Transfer Learning

The model we transferred our data on is the MobileNetV2 model. It was developed by Google in 2017. We chose this model because of its very low parameter count, which was the only model that could be trained under our limited hardware.

Some changes to the image processing were done.

- The images were resized to 224 because MobileNetV2 works up to that value. We also scaled all image values from 0-1 by dividing them by 255 because the weights were trained on values scaled from 0-1
- The blue green red values opencv gave us were transformed into red green blue values because that's what the model was trained on.

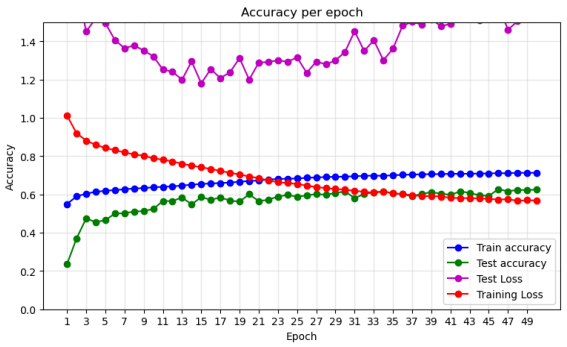


With that said the model structure is very simple at

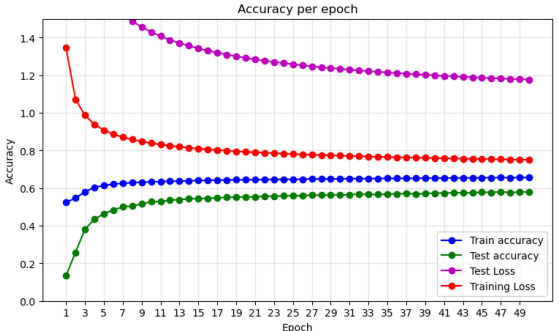
1. Resizing (224,224,Bilinear)
2. MobileNet (Weight: imagenet, with weights frozen)
3. Dense(9, softmax)

VIII. Results

The results of the Custom CNN model comes in with a high increase in accuracy gained per epoch along with a high drop in loss in the beginning but after a while it levels off, gaining a lot less accuracy per epoch later on (around 0.3%). The loss, while it did decrease in change, still did change at a steady enough rate. The unseen data tests per epoch had a similar flow however when the cutoff period was hit the unseen data accuracy did start to reach closer to the training accuracy.



The MobileNetV2 Model performed worse overall with every accuracy score performing worse in the end then the custom model. The losses also never got to a reasonable score. The model hit its limit a lot sooner than the custom model. The unseen data test accuracy vs the training test accuracy still did have their score gap close but a lot slower than the custom model did.



IX. Conclusion

Overall the original model performed much better than the transfer learned model. Getting around 5% more accuracy then the transfer learned model. Both the models looked to be underfitting because of their slow gain in accuracy along with their general lower performance. It's not overfitting because the testing vs training accuracy are closely related to each other though.

X. Future Work

The biggest limitation here was the hardware used. For the most part of the project an old laptop with a weak cpu was used until the ladder half where the training was done on a more modern desktop, but still with the desktop the gpu was unable to properly work with tensorflow.

A possible improvement that could be done especially for the MobileNetV2 model is to not freeze the weights on the MobileNetV2 model but allow for the weights to be trained. This wasn't possible because of the hardware limitations making the model take a very long time per epoch when training also the MobileNetV2 weights.

More image augmentation also could have been done, specifically image stretching, mirroring, and color shifting. Which could possibly improve the level of the period of the model.

Since the model was underfitting and generalizing too much, adding more layers/neurons would be best for future models like these, in order to let the model understand more complex features and potentially score higher.

XI. References

[1] Savya Khosla, "Introduction to Pooling Layer in CNN" GeeksforGeeks. 17 Mar, 2026. https://www.geeksforgeeks.org/deep-learning/introduction-to-pooling-layer-cnn/

[2] Azeem Md, "Applying Batch Normalization using tf.keras.layers.BatchNormalization in Tensorflow", GeeksforGeeks. 17 Mar, 2026 https://www.geeksforgeeks.org/deep-learning/applying-batch-normalization-in-keras-using-batchnormalization-class/

[3] Author Unknown, "Dense layer", keras. https://keras.io/api/layers/core_layers/dense

[4] Rahul R. Kushwaha, "What are Convolution Layers?", GeeksforGeeks. 31 Jul, 2025. https://www.geeksforgeeks.org/machine-learning/what-are-convolution-layers

[5] Author Unknown "What is Mobilenet V2?", GeeksforGeeks. 23 Jul, 2025 https://www.geeksforgeeks.org/computer-vision/what-is-mobilenet-v2

[6] Suman Kunwar and Thierrytheg, "Garbage Dataset", Kaggle. Feb, 2026. https://www.kaggle.com/datasets/sumn2u/garbage-classification-v2